

LEI

Base de Dados

“SIESTÁGIOS”

Gonçalo Sobral N° 129850

Daniel Masqueiro N° 129853

Rafael Silva N° 129834

Turma PL03

2025/2026

Índice

1. Introdução
2. Modelo de Classes de Objetos
 - 2.1. Classes e atributos
 - 2.1.1. Núcleo Utilizadores
 - 2.1.2. Núcleo Escola
 - 2.1.3. Núcleo Empresarial
 - 2.1.4. Núcleo do Estágio
 - 2.1.5. Classes Auxiliares
 - 2.2. Associações, Multiplicidades e Tipos de Relacionamento
 - 2.2.1. Generalização
 - 2.2.2. Composições
 - 2.2.3. Agregações
 - 2.2.4. Associações
3. Modelo Relacional
 - 3.1. Transposição do Diagrama de Classes para o Esquema Relacional
 - 3.1.1. Transposição do Diagrama de Classes para o Esquema Relacional
 - 3.1.2. Transposição da Composição
 - 3.1.3. Transposição de Atributos
 - 3.1.4. Refatoração de Chaves Primárias
 - 3.1.5. Refatoração da Tabela Vagas
 - 3.2. Justificação da Estratégia da Gestão da Integridade Referencial
 - 3.3. Implementação de Triggers
4. Conclusão
 - 4.1. Síntese do Projeto
 - 4.2. Críticas e Pontos de Melhoria
 - 4.2.1. Aspectos Positivos
 - 4.2.2. Aspectos a Melhorar
 - 4.3. Apreciação Final
5. Anexos
 - 5.1. Erros encontrados nas alterações e Otimizações
 - 5.2. Imagens Relevantes

1.Introdução

O projeto realizado tem como objetivo desenvolver uma solução de Base de Dados para um SI (Sistema de Informação) de uma escola profissional ao desenvolver um sistema de apoio para a gestão de estágios fornecido aos seus alunos, denominado “SIEstágios”. Este sistema responde à necessidade de registar, gerir e consultar informações sobre estágios, alunos, formadores, empresas parceiras e estabelecimentos que acolhem estagiários.

O sistema SIEstágios permite:

- Registo e gestão de utilizadores (administrativos, formadores e alunos)
- Gestão de empresas, estabelecimentos e responsáveis por estágios
- Alocação de alunos a locais de estágio e respetivo acompanhamento
- Registo de avaliações (notas de 0 a 20 pela escola/empresa e classificações de 1 a 5 pelos alunos)
- Consulta de locais de estágio por zona geográfica, ramo de atividade, produtos ou transportes disponíveis
- Cálculo automático de médias de avaliação dos estabelecimentos por ano letivo

Neste Caderno iremos descrever a estrutura de dados atribuída para o Sistema. Daqui para a frente iremos abordar 3 aspetos:

1. **Modelo de Classes e Objetos:** Apresenta o Diagrama de Classes UML com as respectivas classes, atributos e ligações/relações do Sistema.
2. **Modelo Relacional:** Detalha a “passagem” do Diagrama de Classes para o Esquema Relacional, onde propomos otimizações para a Gestão da Integridade Referencial.
3. **Dicionário de Dados:** Onde detalhamos cada atributo do modelo relacional, incluindo tipos de dados, restrições e descrições.

2. Modelo de Classes de Objetos

- **Diagrama de Classes (Anexo 6)**

Neste diagrama estão representadas as Classes de Objetos, estas foram desenhadas para representar conceitualmente a Estrutura de Dados do sistema SIEstágios. Este diagrama identifica as principais entidades, atributos, as correlações que existem entre eles bem como as suas multiplicidades, servindo assim como uma “base” para o sistema de base de dados relacional.

2.1 Classes e atributos

2.1.1. Núcleo de Utilizadores

Utilizador: será a classe fundamental para o SI que generaliza o conceito de “pessoa”. Permitirá aos **alunos** e **formadores** navegar no sistema. Cada **utilizador** terá **login**, **password** e **nome**. Dos quais o **login** será o username que será único para cada **utilizador**.

Aluno/Formador/Administrativo: São as explicitações dos utilizadores, um **utilizador** só pode ser ou um **aluno** ou um **formador** ou um **administrativo**, nunca podendo ser os dois ao mesmo tempo. Um aluno de uma escola profissional não será professor ao mesmo tempo que é aluno e o mesmo para o professor não poderá ser aluno. Já o **administrativo** como no enunciado não especifica consideramos que o administrativo será alguém fora da escola que apenas gere o SI registam novos **alunos**, **empresas**, **locais de estágio**, etc. Nas classes **alunos** e **formador** ficará apenas armazenado o **número** de cada um e no caso do **aluno** algumas **observações** e no do **formador** a sua **disciplina**.

2.1.2 Núcleo Escola

Curso: é a classe principal que armazena as turmas por área de estudo. Tendo como atributos a sua **designação** e **código**.

Ano_Letivo: define o período temporal (ex. 2024/2025) em que as turmas operam. Irá permitir que turmas em anos letivos diferentes possam ter a mesma **sigla**. Esta classe também ajudará a funcionar como um histórico, não só das turmas como dos estágios.

Turma: representa a agregação de um grupo de alunos. Esta é a classe a que os alunos se vão ligar diretamente e irá guardar uma **sigla** da qual decidimos funcionar com 3 ou 4 caracteres (ex. 2DA), onde o primeiro dígito representa o ano da turma (1, 2 ou 3), o segundo dígito representa o curso (ex. D - Desporto) e o último dígito é a distinção das turmas do mesmo curso e ano.

2.1.3. Núcleo Empresarial

Empresa: Representa a entidade legal que oferece os estágios. Nela é armazenada a sua **firma, contribuinte, morada, localidade, código postal, telefone, email, website e observações**.

Estabelecimentos: Representam o local físico ou departamento específico onde o estágio ocorre. Modela o facto de uma **Empresa** pode ter múltiplos locais, sendo um estágio alocado a um local específico.

Responsável: Representa o supervisor de estágio da. É ele que vai estar em contacto direto entre o **aluno** dentro da **empresa** para que o possa avaliar e classificá-lo no final. Armazena-se aqui o **nome, título, cargo, telefone, email, telemóvel e observações**. É necessário o nome nesta classe ao contrário das **alunos/formador** já que estes herdam no da classe utilizador é responsável não é um utilizador como descrito no enunciado “Os três tipos de utilizador do sistema (aluno, formador e administrativo)”

2.1.4. Núcleo do Estágio

Estágio: é a classe mais importante, é ela que associa o aluno à empresa, esta associação não poderia ser direta entre ele já que existem atributos próprios como a **data_inicio** e **data_fim**. Também conta com as notas que serão dadas ao aluno, quer seja pelo formador/escola quer seja pelo responsável/empresa.

Vagas: a empresa disponibiliza ou não um **número de vagas** por ano. Esta classe foi feita para armazenar o número de vagas, tendo que ser criada já que a empresa pode ter um número diferente de **vagas** a cada ano. Adicionamos também um atributo de bit de controle **vaga_cheias** para ajudar no futuro a gerir se as vagas para aquela empresa foram preenchidas ou não. As vagas são um número dado pela empresa e quando alguém fizer um pedido para preencher a vaga, a empresa preenche ela e decide para que estabelecimento quer colocar o aluno.

Avaliação Anual Estabelecimento: armazenará a média de notas dadas pelos alunos à empresa de 1 a 5.

2.1.5. Classes auxiliares

Ramo, Zona, Transporte, Produtos: Foram criadas para garantir a normalização. Em vez de permitir que um utilizador escreva o ramo de uma empresa num campo de texto (o que permite erros de digitação), como no exemplo de **Ramo**, ele é forçado a selecionar um objeto da classe **Ramo** para procurar pela área em que quer estagiar.

2.2 Associações, Multiplicidades e Tipos de Relacionamento

2.2.1. Generalização

Utilizador → **Alunos/Formador/Administrativo**: Como foi referido anteriormente as classes aluno, formador e administrativo herdam atributos da classe utilizados por isso ligamos estas classes à utilizador com generalizações.

2.2.2. Composições

Empresas (1) ◆-- (1..*)Estabelecimentos: um estabelecimento pertence a apenas uma empresa. Uma empresa pode ter um ou mais estabelecimentos. Foi criada esta ligação como composição pois se a empresa não existir não faz sentido os estabelecimentos existirem e com a composição ao dar delete na empresas também acaba com os estabelecimentos.

Empresas(1) ◆- (0..*)Vagas: uma empresa pode ter várias vagas ou mesmo nenhuma, mas uma vaga pertence a apenas um empresa. A composição garante que se uma empresa deixar de existir as vagas também passam a não ser úteis por isso dando delete a elas também.

Estabelecimentos(1) ◆- (1..*)Responsável: um estabelecimento terá vários responsáveis já que um pode ter, por exemplo, 15 vagas e assim pode atribuir um responsável a 1 ou mais estágios. Mas se o estabelecimento não existir ninguém pode ser responsável pelo mesmo, ou seja não há necessidade do responsável existir se o estabelecimento também não existir.

2.2.3. Agregações

Turma(1) ◇- (1..*)Aluno: A cardinalidade conceptual foi definida como (1..*) (um ou mais) para o lado do Aluno, indicando que uma turma não pode estar vazia. No entanto, existe uma regra de negócio mais restritiva que especifica: "Cada turma tem entre 10 e 28 alunos". Iremos resolver esta questão de ter de conter entre 10 e 28 alunos mais à frente.

Zona(1) ◇- (0..*) Estabelecimentos: um estabelecimento tem um local fixo ou seja ele terá apenas uma zona, que é a localização geográfica do mesmo. Uma zona pode ter vários estabelecimentos mas esta não é dependente dos mesmos, também pode não haver estabelecimentos na zona.

2.2.4 Associações

Turma(1) <-> (0..*)Curso: Uma turma só pode ser composta por alunos do mesmo curso, mas podem haver várias turmas com o mesmo curso. Decidimos permitir um curso poder ter 0 alunos por que pode haver um ano em que não há inscrições suficientes

para abrir uma turma daquele curso, mas não implica que no futuro não volte a abrir turmas do mesmo.

Turma(1) <-> (1..*)AnoLetivo: as turmas funcionam em apenas um ano letivo e no mesmo ano letivo podem existir várias turmas.

Alunos(0..*) <-> (1)Administrativo: um administrativo pode criar vários alunos ou nenhum e um aluno só pode ter sido criado por um administrativo.

Formador(1) <-> (0..*)Estagio: um formador pode avaliar e supervisionar vários estágios, mas um estágio só pode ser supervisionado por um formador

Alunos(1) <-> (1..*)Estagio: um aluno pode fazer vários estágios, mas um estágio só pode ser realizado por um aluno, já que é em estágio que está a avaliação do mesmo.

Responsavel(1) <-> (0..*)Estagio: um responsável pode supervisionar vários estágios ou nenhum e um estágio só pode ser supervisionado por um responsável.

Estagio(1) <-> (0..*)Estabelecimentos: um estágio será realizado num só estabelecimento, mas vários estágios podem ser realizados no mesmo estabelecimento.

Administrativo(1) <-> (0..*)Estabelecimentos: um administrativo pode registar vários estabelecimentos, ou nenhum, um estabelecimento só pode ter sido registado por um administrativo.

Administrativo(1) <-> (0..*)Empresas: um administrativo pode registar várias, ou nenhuma, empresas e um empresa só pode ter sido registada por um administrativo.

Administrativo(1) <-> (0..*)Ramo: um administrativo pode registar vários, ou nenhum, ramos e um ramo só pode ter sido registado por um administrativo.

Empresas(0..*) <-> (1..*)Ramo: empresas podem ter vários ramos de atividade tendo pelo menos um, e varias empresas podem ter o mesmo ramo de atividade.

AnoLetivo(1) <-> (0..*)AvaliacaoAnualEstabelecimentos: Multiplicidade-uma avaliação anual só pode ser de um ano letivo e num ano letivo pode ter ocorrido várias avaliações de diferentes estabelecimentos.

Estabelecimentos(0..*) <-> (1..*)Produtos: um estabelecimento tem pelo menos um produto principal, mas pode ter vários. Um produto pode ser oferecido por outros estabelecimentos.

Estabelecimentos(0..*) <-> (0..*)Transporte: um estabelecimento pode ter ao seu alcance vários transportes ou nenhum, e um transporte pode estar ao alcance de vários estabelecimentos diferentes.

Estabelecimentos(1) <-> (0..*)AvaliacaoAnualEstabelecimentos: um estabelecimento vai ser avaliado vários anos ou não se não disponibilizar vagas. Uma avaliação será apenas de um estabelecimento.

Zona(1..*) <-> (0..*)Transporte: uma zona pode dispor de vários transportes diferentes ou de nenhum, mas um transporte fornece pelo menos uma zona e pode abranger várias.

AnoLetivo(1) <-> (0..*)Vagas: determinadas vagas são só para um ano letivo, já em um ano letivo podem existir várias vagas de diferentes empresas.

Empresas(1) <-> (0..*)Vagas: uma empresa pode ter várias vagas em diferentes anos e vagas pertencem apenas a uma empresa.

3. Modelo Relacional

Após o UmLToRel gerar os comandos DDL foi gerado um modelo relacional no phpMyAdmin (Anexo 7)

3.1 Transposição do Diagrama de Classes para o Esquema Relacional

Mas este tem algumas diferenças de como criamos implementar a Base de Dados. As principais decisões quanto à transposição tomadas para fazer este esquema relacional foram:

3.1.1 Transposição da Generalização

Antes: A ferramenta transpôs a herança. Criou a tabela "mãe" **utilizador** e as tabelas "filhas" **aluno**, **formador** e **administrativo**. As chaves primárias (PK) das tabelas filhas (ex: **aluno.utilizador_utilizador_ID**) foram definidas como Chaves Estrangeiras (FK) a apontar para a **utilizador**.

Depois: A estrutura foi mantida, mas os nomes das colunas foram normalizados (ex: **utilizador_utilizador_ID** passou a **utilizador_ID**) para simplificar.

3.1.2. Transposição da Composição (Empresa, Estabelecimento e Vagas)

A ferramenta gerou uma Chave Primária Composta para **estabelecimentos** (PK(**empresas_empresas_ID**, **estabelecimentos_ID**)), com uma FK a ligar **empresas_empresas_ID** à tabela **empresas**.

Depois: Esta lógica de transposição estava correta e foi mantida, pois implementa perfeitamente a regra de que um Estabelecimento é identificado unicamente *dentro* da sua Empresa e que pretendemos implementar.

Ao fazer esta alteração deparamo-nos com o **Erro #1553** que não nos permitia eliminar a PK composta. Ao analisar a base de dados reparamos que não o conseguimos fazer já que havia FK compostas apontando para a FK original. Para o resolver tivemos de remover todas as FK que referenciam estabelecimentos, alterar a PK de estabelecimentos e depois alterar a estrutura das tabelas dependentes a estabelecimentos.

3.1.3. Transposição de Atributos (Otimização de Tipos de Dados)

Antes: A geração automática usou tipos de dados genéricos e não otimizados. Chaves primárias eram integer simples, e a maioria dos campos de texto (como **nome**, **login**, **email**) foram criados como **TEXT**.

Depois: Ocorreu um refinamento manual crucial:

Chaves Primárias (PKs): Foram alteradas para **INT NOT NULL AUTO_INCREMENT** (ex: em **utilizador**, **empresas**, **estagio**). Isto automatiza a geração de IDs únicos, garante que não são nulos e simplifica os comandos **INSERT**.

Texto: Campos **TEXT** genéricos foram substituídos por **VARCHAR(N) NOT NULL** (ex: **codigo_postal VARCHAR**). Isto melhora drasticamente o desempenho da indexação e das pesquisas, ao mesmo tempo que garante a integridade dos dados (campos obrigatórios).

3.1.4. Refatoração de Chaves Primárias (PKs)

Antes: A maioria das PKs eram integer simples. A tabela **responsavel** foi transposta com uma PK composta de 3 colunas (PK(**estabelecimentos...**, **estabelecimentos...**, **responsavel_ID**)), herdada da sua classe "mãe" **estabelecimentos**.

Depois: Ocorreu um refinamento manual crucial:

PKs Simples: A maioria das PKs (**utilizador_ID**, **curso_ID**, **estagio_ID**, etc.) foi alterada para **INT NOT NULL AUTO_INCREMENT**, automatizando a geração de IDs únicos.

PK de responsável: A tabela **responsável** foi refatorada para ter a sua própria PK removendo assim a chave composta por três colunas — **id_responsavel** (PK, AI).

Justificação: Esta refatoração foi crucial. O modelo original forçava qualquer tabela que referenciasse **responsável** (como **estágio**) a usar *três colunas* para a FK. Ao mudar para uma PK simples (**id_responsavel**), a referência na tabela **Estágio** foi simplificada para *uma única coluna* (**responsavel_id_responsavel**), tornando o esquema mais limpo e eficiente.

Ao fazer esta melhoria deparamo-nos com o **Erro #1025** que é parecido com o erro que tivemos anteriormente com os **estabelecimentos**, tivemos de remover a FK da tabela **Estagio**, alterar o **Responsável** e **Estagio** e voltar a criar as FK.

3.1.5. Refatoração da Tabela vagas (Chave Natural)

Antes: A tabela usava uma PK composta ineficiente (PK(empresas_empresas_ID, vagas_ID)), onde **vagas_ID** era uma chave artificial que não impedia a duplicação de registos para a mesma empresa no mesmo ano.

Depois: A coluna **vagas_ID** foi eliminada. A PK foi substituída por uma Chave Primária natural e composta: PK(empresas_empresas_ID, ano_letivo_ano_letivo_ID).

Justificação: Esta nova PK garante a regra de negócio (uma empresa só pode ter um registo de vagas por ano) ao nível do motor da base de dados. A coluna **vagas_cheias** foi mantida deliberadamente como uma otimização de performance para a aplicação.

3.2 Justificação da Estratégia de Gestão da Integridade Referencial

A integridade referencial (a garantia de que as ligações entre tabelas são sempre válidas) é assegurada pela implementação rigorosa de Chaves Estrangeiras (FKs).

Antes: O script gerado pelo XML era inconsistente. Na tabela estagio, existiam *cinco* colunas de FKs para locais: duas para estabelecimentos e três para responsavel (que por sua vez já apontava para estabelecimentos).

Depois: A estratégia foi totalmente reescrita manualmente:

1. Correção de Redundância (Otimização 1): As colunas **estabelecimento_empresa_ID** e **estabelecimento_ID** foram *removidas* da tabela estágio. A ligação ao local do estágio é agora obtida *através* da **responsavel_id_responsavel** (JOIN estágio -> responsável -> estabelecimentos). Isto garante uma Single Source of Truth e impede dados contraditórios.
2. Correção de Duplicação: A FK **responsavel_estabelecimentos** duplicada na tabela **responsável** foi eliminada.
3. Estratégia ON UPDATE: Foi generalizado o **ON UPDATE CASCADE** para permitir flexibilidade se as PKs forem alteradas.
4. Estratégia ON DELETE: Foi usada **CASCADE** para relações de composição (ex: Empresa -> estabelecimentos) e **RESTRICT** (implícito) para associações (ex: Turma -> aluno), para proteger dados críticos de serem apagados.

Após estas atualizações ficamos com um modelo relacional mais simplificado e mais eficiente (**Anexo 8**).

3.3 Implementação de triggers

O ficheiro gerado pelo XMI representa o esqueleto estrutural "passivo".

Otimização 1: Integridade Comportamental (Triggers de Regra de Negócio)

Antes: Inexistente. O ficheiro não continha Triggers.

Depois: Foram adicionados:

Para limitar o número máximo de alunos por turma: **trg_check_turma_max_before_insert (Anexo 9)** (limite de 28), que ao dar **INSERT** de um aluno em uma turma verifica se ela já tem 28 alunos ou não se já estiver devolve um erro dizendo que a turma já está cheia.

Para limitar o número mínimo de alunos por turma: **trg_check_turma_min_before_delete (Anexo 10)** (mínimo de 10), que ao dar de um aluno em uma turma verifica se ela já tem 28 alunos ou não se já estiver devolve um erro dizendo que a turma já está cheia. Ao dar **DELETE FROM** irá verificar se a turma tem 10 alunos ou menos e não irá permitir remover o aluno da turma.

Para complementar as anteriores foi também implementado o trigger **trg_check_turma_update_before_update (Anexo 20)**.

Para limitar as notas em estágio **trg_check_notas_before_insert (Anexo 11)** e **trg_check_notas_before_update (Anexo 17)** (validar 0-20).

Otimização 2: Integridade da Hierarquia

Antes: Após alguns testes verificamos que era permitido que um **utilizador_ID** fosse **Aluno** e **Formador** ao mesmo tempo e isto não era suposto ocorrer.

Depois: Adicionamos os triggers **trg_check_disjoint_..._before_insert (Anexo 21, 22, 23)**, a cada subclasse (**alunos**, **formador** e **administrativo**) e que verifica que ao inserir um utilizador numa destas subclasses verifica se o **utilizador** já está presente em uma outra classe, se já estiver devolve um erro já que não podem ser ambos ao mesmo tempo (ex. um aluno não pode ser formador).

Otimização 3: Integridade Lógica (Estágio vs Responsável)

Antes: Ao realizarmos mais alguns testes verificamos que era possível colocar um responsável num estágio com um estabelecimento diferente, ou seja, o estágio seria supervisionado por um responsável que não era o do estabelecimento que estava associado ao estágio

Depois: Adicionados `trg_check_responsavel_estabelecimento_....` (Anexo 18, 19) no `INSERT` e com `UPDATE`, que garante que o responsável pertence ao estabelecimento associado ao estágio.

Justificação: Após a Otimização 1, surgiu um novo desafio: garantir que o responsável escolhido para o estágio pertencia ao estabelecimento correto. Este trigger foi a solução: ele verifica se o responsável pertence ao estabelecimento do estágio, impedindo dados logicamente inconsistentes.

Otimização 4: Automação de Cálculos (Nota Final)

Antes: `nota_final` era um campo manual.

Depois: Adicionados os triggers `trg_calcular_nota_final....` (Anexo 15, 16) para automatizar o processo de atribuição de notas. O formador e o responsável vão inserir uma nota dos determinados campos e à medida que vão sendo atribuídos valores às colunas a coluna `nota_final` vai atualizando de acordo com

Justificação: Garante a integridade dos dados. A `nota_final` não pode ser inserida manualmente; é sempre calculada automaticamente com base nos atributos. Isto é uma forma de melhorar o desempenho das leituras.

Otimização 5: Automação de Agregação (Média Estabelecimento)

Antes: Inexistente. A `media_avaliacoes` era um campo manual.

Depois: Adicionado o `PROCEDURE sp_atualizar_media_estabelecimento` (Anexo 24) e os `TRIGGERS` (`trg_atualizar_media_estab_....`) (Anexo 12, 13, 14) na tabela `estagio` para o chamar.

Justificação: Esta é a otimização mais avançada do sistema. Ela garante que a `media_avaliacoes` (que é de 1-5) na tabela `avaliacao_anual_estabelecimento` é sempre um reflexo em tempo real da média das `nota_dada_pelo_aluno` na tabela `estagio`. Isto automatiza um cálculo complexo e garante que os relatórios de avaliação estão sempre corretos.

Otimização 6: Tipos de dados

Para garantir a coerência e lógica dos dados realizamos algumas alterações às colunas de maior parte das classes.

Numero: nas tabelas `alunos` e `formador` colocamos a coluna `numero` como `AUTO INCREMENT`, `NOT NULL` e `UNIQUE`. Um aluno e um formador têm que ter um número portanto não pode ser null, os números dos alunos e dos formadores serão únicos, ou seja, dois alunos não podem ter o mesmo número e o mesmo com os formadores, dois formadores não podem ter o mesmo número.

AnoLetivo: mudamos o tipo de variável da coluna `ano` para `VARCHAR(11)` pois irá conter apenas (ex. 2024/2025) que será apenas 11 caracteres. Este também será `UNIQUE`.

Também terá que ser **NOT NULL**, pois será preenchido obrigatoriamente. Ao realizar esta alteração deparamo-nos com o **Erro #1064** que rapidamente corrigimos pois apenas faltava colocar o comprimento da variável.

Curso: o código do curso também será **UNIQUE**.

Empresas e estabelecimentos: contribuinte, email, website e telefone serão **UNIQUE** por razões óbvias, o contribuinte é único para cada empresa, um website não pode ser para várias empresas, e um email só pode pertencer a uma empresa, o mesmo com o telefone. O mesmo ocorre com o **telefone** e **email** do estabelecimento.

Datas: também mudamos todas as datas para **DATE** ou **DATETIME**, como **dada_inicio**, **data_fim**, **data_fundação**, etc.

Responsável: será a mesma lógica das empresas, terá o seu **telefone**, **telemovel** e **email** como **UNIQUE**.

Utilizador: login será **UNIQUE**, pois pessoas diferentes não podem ter o mesmo username, isso daria muitos problemas. Ao realizar esta alteração deparamo-nos com o **Erro #1834** - não reparamos que tínhamos criado 2 utilizadores com o mesmo **login**, precisamos de os alterar para que login pudesse ser **UNIQUE**.

Ramo: **CAE** foi mudado para **INT** ao procurar percebemos que este código é composto apenas por números inteiros. Este valor também será **NOT NULL** pois tem que ser preenchido obrigatoriamente.

Foram também alteradas várias colunas para que tivessem que ser preenchidas obrigatoriamente com a **NOT NULL**, como:

Curso -> codigo

Empresas -> contribuinte, email, telefone, codigo_postal, morada, firma e localidade. Ao fazer esta alteração deparamo-nos com o **Erro #1048** - que é similar a #1834, apenas tínhamos uma empresa com o campo morada como **NULL**. Para corrigir o erro apenas adicionamos uma morada a esse campo.

Produtos -> nome, marca e tipo_produto

Responsável -> nome, título, cargo, telefone, email e telemovel

Transporte -> meio_transporte e linha

Turma -> sigla

Utilizador: nome, login e password.

Zona -> localidade e designação

Nota: Em **Formador** decidimos deixar a disciplina como **NULL**, pois existem formadores que não dão aulas de nenhuma disciplina mas como têm outras funções na escola podem ser atribuídos para avaliar certo estágio.

4. Conclusão

4.1 Síntese do Projeto

O projeto que realizamos (SIEstágios) constitui uma solução para a gestão de estágios de uma escola profissional, tendo sido desenvolvido com base nos requisitos especificados no enunciado. A base de dados implementada contém todas as entidades essenciais para o funcionamento do sistema: utilizadores (administrativos, alunos e formadores), empresas, estabelecimentos, estágios, cursos, turmas, zonas, transportes e as suas respetivas relações entre elas.

Foi implementado com sucesso os seguintes pontos do enunciado:

- Gestão completa de utilizadores (administrativo, aluno ou formador);
- Controlo da composição das turmas (mínimo 10, máximo 28 alunos);
- Registo detalhado de empresas, estabelecimentos e responsáveis;
- Acompanhamento integral dos estágios, incluindo componentes de avaliação;
- Cálculo automático de médias de avaliação dos estabelecimentos por ano letivo e da nota final do aluno em cada estágio;
- Sistema de consulta por zonas, transportes, ramos de atividade e produtos.

4.2 Críticas e Pontos de Melhoria

4.2.1 Aspectos Positivos

A implementação usa triggers para garantir restrições complexas que não podiam ser implementadas apenas com foreign keys, nomeadamente:

- Especialização da classe utilizador entre administrativo, aluno e formador;
- Limite mínimo e máximo de alunos por turma;
- Validação de notas, ou seja, limita-las entre 1-20 ou 1-5;
- Cálculo automático da nota final do estágio.

O sistema implementa um procedimento e triggers que automatizam processos, como:

- Cálculo da média das avaliações dos estabelecimentos (`sp_atualizar_media_estabelecimento`);

- Atualização automática das médias após inserção, atualização ou eliminação de estágios

- Validação da relação entre responsáveis e estabelecimentos

O modelo permite expansão futura, com campos de observações em praticamente todas as entidades principais, possibilitando guardar informação adicional sem necessidade de alterações estruturais.

4.2.2 Aspectos a Melhorar

1. Tipo de Dados Inadequados

Campos de telefone definidos como **INT** pois não permitem prefixos internacionais (ex: +351). Uma possível solução seria utilizar **VARCHAR(20)** para maior flexibilidade, mas decidimos não implementar isto, pois não garante que sejam escritos apenas números.

2. Validação de Datas

- Não há validação de que `data_fim` é posterior a `data_inicio` nos estágios

- Não há validação de que `data_fundacao` dos estabelecimentos é anterior à data atual

Decidimos não integrar estas validações nesta fase, confiando na correta inserção de dados pelo administrativo e formador. No entanto, os problemas encontrados ao testar a inserção de estágios em anos letivos passados levaram-nos a priorizar o desenvolvimento desta funcionalidade na próxima parte do projeto.

4.3 Apreciação Final

No projeto usamos os conceitos de bases de dados e implementamos aquilo pedido no enunciado, desde a estrutura criada responde adequadamente aos requisitos especificados, como as funcionalidades descritas no enunciado foram implementadas e o uso adequado de constraints, triggers e procedimentos para garantir o que era pedido no enunciado.

5. Anexos

5.1 Erros encontrados nas alterações e Otimizações

Anexo 1 - Erro #1064:

 Erro de consulta:

#1064 - Você tem um erro de sintaxe no seu SQL próximo a 'CHARACTER SET latin1 COLLATE latin1_bin NOT NULL' na linha 1

Anexo 2 - Erro #1553:

Mensagens do MySQL : 

#1553 - Cannot drop index 'PRIMARY': needed in a foreign key constraint

Anexo 3 - Erro #1025:

#1025 - Erro ao renomear './erros\#sql-44f4_c1' para './erros\responsavel' (erro no. 150 "Foreign key constraint is incorrectly formed")

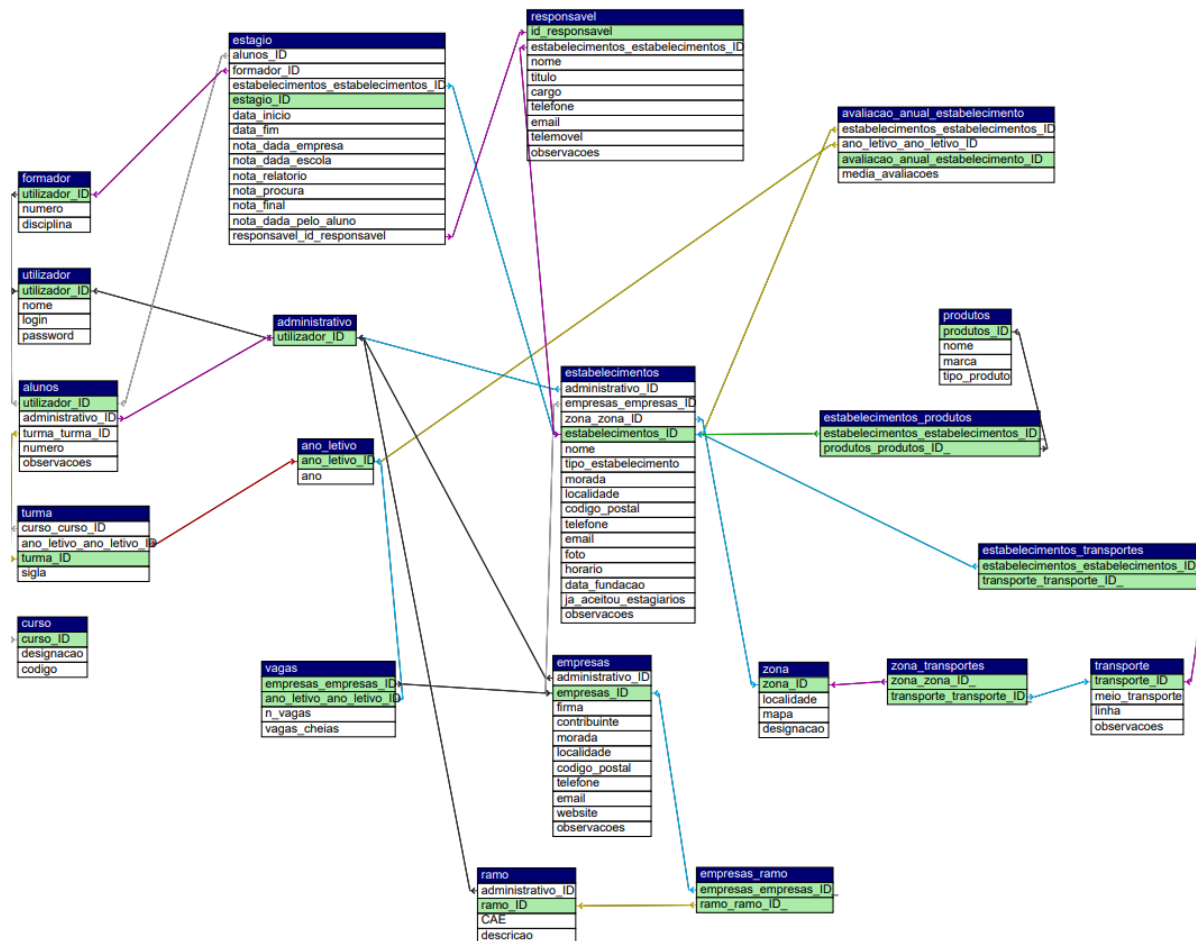
Anexo 4 - Erro #1834:

#1834 - Cannot delete rows from table which
is parent in a foreign key constraint
'FK_administrativo_utilizador' of table
'administrativo'

Anexo 5 - Erro #1048:

#1048 - Coluna 'morada' não pode ser vazia

Anexo 8 - Modelo relacional após otimizações e alterações:



Procedimento e Triggers

Anexo 9 - trg_check_turma_max_before_insert

```
BEGIN
  DECLARE v_count INT;
  SELECT COUNT(*) INTO v_count FROM `alunos` WHERE `turma_turma_ID` =
  NEW.turma_turma_ID;
  IF v_count >= 28 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A turma já atingiu o limite
    máximo de 28 alunos.';
  END IF;
END
```

Anexo 10 - trg_check_turma_min_before_delete

```
BEGIN
  DECLARE v_count INT;
  SELECT COUNT(*) INTO v_count FROM `alunos` WHERE `turma_turma_ID` =
OLD.turma_turma_ID;
  IF v_count <= 10 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A turma não pode ficar com menos
de 10 alunos.';
  END IF;
END
```

Anexo 11 - trg_check_notas_before_insert

```
BEGIN
  IF NEW.nota_dada_empresa IS NOT NULL AND (NEW.nota_dada_empresa < 0 OR
NEW.nota_dada_empresa > 20) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_dada_empresa está fora do
intervalo (0 a 20).';
  END IF;
  IF NEW.nota_dada_escola IS NOT NULL AND (NEW.nota_dada_escola < 0 OR
NEW.nota_dada_escola > 20) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_dada_escola está fora do
intervalo (0 a 20).';
  END IF;
  IF NEW.nota_relatorio IS NOT NULL AND (NEW.nota_relatorio < 0 OR NEW.nota_relatorio
> 20) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_relatorio está fora do
intervalo (0 a 20).';
  END IF;
  IF NEW.nota_procura IS NOT NULL AND (NEW.nota_procura < 0 OR NEW.nota_procura >
20) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_procura está fora do
intervalo (0 a 20).';
  END IF;
  IF NEW.nota_dada_pelo_aluno IS NOT NULL AND (NEW.nota_dada_pelo_aluno < 1 OR
NEW.nota_dada_pelo_aluno > 5) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_dada_pelo_aluno está fora
do intervalo (1 a 5).';
  END IF;
END
```

Anexo 12 - trg_atualizar_media_estab_after_delete

```
BEGIN
  DECLARE v_ano_letivo_id INT;

  IF OLD.nota_dada_pelo_aluno IS NOT NULL THEN
    SELECT t.ano_letivo_ano_letivo_ID INTO v_ano_letivo_id
    FROM `alunos` a
    JOIN `turma` t ON a.turma_turma_ID = t.turma_ID
    WHERE a.utilizador_ID = OLD.alunos_ID;

    IF v_ano_letivo_id IS NOT NULL THEN
      CALL sp_atualizar_media_estabelecimento(
        OLD.estabelecimentos_estabelecimentos_ID,
        v_ano_letivo_id
      );
    END IF;
  END IF;
END
```

Anexo 13 - trg_atualizar_media_estab_after_insert

```
BEGIN
  DECLARE v_ano_letivo_id INT;

  IF NEW.nota_dada_pelo_aluno IS NOT NULL THEN
    SELECT t.ano_letivo_ano_letivo_ID INTO v_ano_letivo_id
    FROM `alunos` a
    JOIN `turma` t ON a.turma_turma_ID = t.turma_ID
    WHERE a.utilizador_ID = NEW.alunos_ID;

    IF v_ano_letivo_id IS NOT NULL THEN
      CALL sp_atualizar_media_estabelecimento(
        NEW.estabelecimentos_estabelecimentos_ID,
        v_ano_letivo_id
      );
    END IF;
  END IF;
END
```

Anexo 14 - trg_atualizar_media_estab_after_update

```
BEGIN
  DECLARE v_ano_letivo_id INT;
  DECLARE v_ano_letivo_id_old INT;

  IF (NEW.nota_dada_pelo_aluno != OLD.nota_dada_pelo_aluno OR
      NEW.alunos_ID != OLD.alunos_ID OR
      NEW.estabelecimentos_estabelecimentos_ID !=
      OLD.estabelecimentos_estabelecimentos_ID) THEN

    SELECT t.ano_letivo_ano_letivo_ID INTO v_ano_letivo_id
    FROM `alunos` a
    JOIN `turma` t ON a.turma_turma_ID = t.turma_ID
    WHERE a.utilizador_ID = NEW.alunos_ID;

    IF v_ano_letivo_id IS NOT NULL THEN
      CALL sp_atualizar_media_estabelecimento(
        NEW.estabelecimentos_estabelecimentos_ID,
        v_ano_letivo_id
      );
    END IF;

    IF (NEW.alunos_ID != OLD.alunos_ID OR
        NEW.estabelecimentos_estabelecimentos_ID != OLD.estabelecimentos_ID) THEN

      SELECT t.ano_letivo_ano_letivo_ID INTO v_ano_letivo_id_old
      FROM `alunos` a
      JOIN `turma` t ON a.turma_turma_ID = t.turma_ID
      WHERE a.utilizador_ID = OLD.alunos_ID;

      IF v_ano_letivo_id_old IS NOT NULL THEN
        CALL sp_atualizar_media_estabelecimento(
          OLD.estabelecimentos_estabelecimentos_ID,
          v_ano_letivo_id_old
        );
      END IF;
    END IF;
  END IF;
END
```

Anexo 15 - trg_calcular_nota_final_before_insert

```
BEGIN
  DECLARE v_soma DECIMAL(10, 2);
  DECLARE v_contador INT;
  SET v_soma = 0;
  SET v_contador = 0;
  IF NEW.nota_dada_empresa IS NOT NULL THEN
    SET v_soma = v_soma + NEW.nota_dada_empresa;
    SET v_contador = v_contador + 1;
  END IF;
  IF NEW.nota_dada_escola IS NOT NULL THEN
    SET v_soma = v_soma + NEW.nota_dada_escola;
    SET v_contador = v_contador + 1;
  END IF;
  IF NEW.nota_relatorio IS NOT NULL THEN
    SET v_soma = v_soma + NEW.nota_relatorio;
    SET v_contador = v_contador + 1;
  END IF;
  IF NEW.nota_procura IS NOT NULL THEN
    SET v_soma = v_soma + NEW.nota_procura;
    SET v_contador = v_contador + 1;
  END IF;
  IF v_contador > 0 THEN
    SET NEW.nota_final = v_soma / v_contador;
  ELSE
    SET NEW.nota_final = NULL;
  END IF;
END
```

Anexo 16 - trg_calcular_nota_final_before_update

```
BEGIN
  DECLARE v_soma DECIMAL(10, 2);
  DECLARE v_contador INT;
  SET v_soma = 0;
  SET v_contador = 0;
  IF NEW.nota_dada_empresa IS NOT NULL THEN
    SET v_soma = v_soma + NEW.nota_dada_empresa;
    SET v_contador = v_contador + 1;
  END IF;
  IF NEW.nota_dada_escola IS NOT NULL THEN
    SET v_soma = v_soma + NEW.nota_dada_escola;
    SET v_contador = v_contador + 1;
  END IF;
  IF NEW.nota_relatorio IS NOT NULL THEN
    SET v_soma = v_soma + NEW.nota_relatorio;
    SET v_contador = v_contador + 1;
  END IF;
  IF NEW.nota_procura IS NOT NULL THEN
    SET v_soma = v_soma + NEW.nota_procura;
    SET v_contador = v_contador + 1;
  END IF;
  IF v_contador > 0 THEN
    SET NEW.nota_final = v_soma / v_contador;
  ELSE
    SET NEW.nota_final = NULL;
  END IF;
END
```

Anexo 17 - trg_check_notas_before_update

```
BEGIN
  IF NEW.nota_dada_empresa IS NOT NULL AND (NEW.nota_dada_empresa < 0 OR
NEW.nota_dada_empresa > 20) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_dada_empresa está fora do
intervalo (0 a 20).';
  END IF;
  IF NEW.nota_dada_escola IS NOT NULL AND (NEW.nota_dada_escola < 0 OR
NEW.nota_dada_escola > 20) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_dada_escola está fora do
intervalo (0 a 20).';
  END IF;
  IF NEW.nota_relatorio IS NOT NULL AND (NEW.nota_relatorio < 0 OR NEW.nota_relatorio
> 20) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_relatorio está fora do
intervalo (0 a 20).';
  END IF;
  IF NEW.nota_procura IS NOT NULL AND (NEW.nota_procura < 0 OR NEW.nota_procura >
20) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_procura está fora do
intervalo (0 a 20).';
  END IF;
  IF NEW.nota_dada_pelo_aluno IS NOT NULL AND (NEW.nota_dada_pelo_aluno < 1 OR
NEW.nota_dada_pelo_aluno > 5) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A nota_dada_pelo_aluno está fora
do intervalo (1 a 5).';
  END IF;
END
```

Anexo 18 - trg_check_responsavel_estabelecimento_before_insert

```
BEGIN
  DECLARE v_resp_estab_id INT;

  SELECT `estabelecimentos_estabelecimentos_ID`
  INTO v_resp_estab_id
  FROM `responsavel`
  WHERE `id_responsavel` = NEW.responsavel_id_responsavel;

  IF (v_resp_estab_id != NEW.estabelecimentos_estabelecimentos_ID) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Erro: O responsável selecionado não pertence ao
estabelecimento de estágio especificado.';
  END IF;
END
```

Anexo 19 - trg_check_responsavel_estabelecimento_before_update

```
BEGIN
  DECLARE v_resp_estab_id INT;

  SELECT `estabelecimentos_estabelecimentos_ID`
  INTO v_resp_estab_id
  FROM `responsavel`
  WHERE `id_responsavel` = NEW.responsavel_id_responsavel;

  IF (v_resp_estab_id != NEW.estabelecimentos_estabelecimentos_ID) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Erro: O responsável selecionado não pertence ao
estabelecimento de estágio especificado.';
  END IF;
END
```

Anexo 20 - trg_check_turma_update_before_update

```
BEGIN
  DECLARE v_count_new INT;
  DECLARE v_count_old INT;
  IF OLD.turma_turma_ID != NEW.turma_turma_ID THEN
    SELECT COUNT(*) INTO v_count_new FROM `alunos` WHERE `turma_turma_ID` =
NEW.turma_turma_ID;
    IF v_count_new >= 28 THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A turma de destino está cheia
(28 alunos).';
    END IF;
    SELECT COUNT(*) INTO v_count_old FROM `alunos` WHERE `turma_turma_ID` =
OLD.turma_turma_ID;
    IF v_count_old <= 10 THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A turma de origem não pode
ficar com menos de 10 alunos.';
    END IF;
  END IF;
END
```

Anexo 21 - trg_check_disjoint_admin_before_insert

```
BEGIN
  DECLARE v_is_aluno INT;
  DECLARE v_is_formador INT;
  SELECT COUNT(*) INTO v_is_aluno FROM `alunos` WHERE `utilizador_ID` =
NEW.utilizador_ID;
  SELECT COUNT(*) INTO v_is_formador FROM `formador` WHERE `utilizador_ID` =
NEW.utilizador_ID;
  IF (v_is_aluno > 0 OR v_is_formador > 0) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: Este utilizador já existe como
aluno ou formador.';
  END IF;
END
```

Anexo 22 - trg_check_disjoint_aluno_before_insert

```
BEGIN
  DECLARE v_is_formador INT;
  DECLARE v_is_admin INT;
  SELECT COUNT(*) INTO v_is_formador FROM `formador` WHERE `utilizador_ID` =
NEW.utilizador_ID;
  SELECT COUNT(*) INTO v_is_admin FROM `administrativo` WHERE `utilizador_ID` =
NEW.utilizador_ID;
  IF (v_is_formador > 0 OR v_is_admin > 0) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: Este utilizador já existe como
formador ou administrativo.';
  END IF;
END
```

Anexo 23 - trg_check_disjoint_formador_before_insert

```
BEGIN
  DECLARE v_is_aluno INT;
  DECLARE v_is_admin INT;
  SELECT COUNT(*) INTO v_is_aluno FROM `alunos` WHERE `utilizador_ID` =
NEW.utilizador_ID;
  SELECT COUNT(*) INTO v_is_admin FROM `administrativo` WHERE `utilizador_ID` =
NEW.utilizador_ID;
  IF (v_is_aluno > 0 OR v_is_admin > 0) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: Este utilizador já existe como
aluno ou administrativo.';
  END IF;
END
```

Anexo 24 - sp_atualizar_media_estabelecimento**BEGIN***DECLARE v_media DECIMAL(10, 2);**DECLARE v_avaliacao_id INT;**SELECT AVG(e.nota_dada_pelo_aluno) INTO v_media**FROM estagio e**JOIN alunos a ON e.alunos_ID = a.utilizador_ID**JOIN turma t ON a.turma_turma_ID = t.turma_ID**WHERE e.estabelecimentos_estabelecimentos_ID = p_estabelecimento_id**AND t.ano_letivo_ano_letivo_ID = p_ano_letivo_id**AND e.nota_dada_pelo_aluno IS NOT NULL;**SELECT avaliacao_anual_estabelecimento_ID INTO v_avaliacao_id**FROM avaliacao_anual_estabelecimento**WHERE estabelecimentos_estabelecimentos_ID = p_estabelecimento_id**AND ano_letivo_ano_letivo_ID = p_ano_letivo_id**LIMIT 1;**IF v_media IS NOT NULL THEN**IF v_avaliacao_id IS NOT NULL THEN**UPDATE avaliacao_anual_estabelecimento**SET media_avaliacoes = v_media**WHERE avaliacao_anual_estabelecimento_ID = v_avaliacao_id;**ELSE**SELECT IFNULL(MAX(avaliacao_anual_estabelecimento_ID), 0) + 1**INTO v_avaliacao_id**FROM avaliacao_anual_estabelecimento;**INSERT INTO avaliacao_anual_estabelecimento (**estabelecimentos_estabelecimentos_ID,**ano_letivo_ano_letivo_ID,**avaliacao_anual_estabelecimento_ID,**media_avaliacoes**) VALUES (**p_estabelecimento_id,**p_ano_letivo_id,**v_avaliacao_id,**v_media**);**END IF;**END IF;***END****Anexo 25- Dicionário de dados**

siestagios

administrativo

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
utilizador_ID (<i>Primária</i>)	int(11)	Não		utilizador -> utilizador_ID		

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	utilizador_ID	4	A	Não	

alunos

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
utilizador_ID (<i>Primária</i>)	int(11)	Não		utilizador -> utilizador_ID		
administrativo_ID	int(11)	Não		administrativo -> utilizador_ID		
turma_turma_ID	int(11)	Não		turma -> turma_ID		
numero	int(11)	Não				
observacoes	text	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	utilizador_ID	350	A	Não	
numero	BTREE	Sim	Não	numero	350	A	Não	
FK_alunos_turma_alunos_turma	BTREE	Não	Não	turma_turma_ID	38	A	Não	
FK_alunos_administrativo	BTREE	Não	Não	administrativo_ID	2	A	Não	

ano_letivo

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
ano_letivo_ID (<i>Primária</i>)	int(11)	Não				
ano	varchar(11)	Não				

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	ano_letivo_ID	2	A	Não	
ano	BTREE	Sim	Não	ano	2	A	Não	

avaliacao_anual_estabelecimento

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
estabelecimentos_estabelecimentos_ID	int(11)	Não		estabelecimentos -> estabelecimentos_ID		
ano_letivo_ano_letivo_ID	int(11)	Não		ano_letivo -> ano_letivo_ID		
avaliacao_anual_estabelecimento_ID (<i>Primária</i>)	int(11)	Não				
media_avaliacoes	int(11)	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	avaliacao_anual_estabelecimento_ID	19	A	Não	
FK_avaliacao_anual_estabelecimento_noname_estabelecimentos	BTREE	Não	Não	estabelecimentos_estabelecimentos_ID	19	A	Não	
FK_avaliacao_anual_estabelecimento_noname_ano_letivo	BTREE	Não	Não	ano_letivo_ano_letivo_ID	4	A	Não	

curso

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
curso_ID (<i>Primária</i>)	int(11)	Não				
designacao	text	Não				
codigo	int(11)	Não				

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	curso_ID	3	A	Não	
codigo	BTREE	Sim	Não	codigo	3	A	Não	

empresas

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
administrativo_ID	int(11)	Não		administrativo -> utilizador_ID		
empresas_ID (Primária)	int(11)	Não				
firma	text	Não				
contribuinte	int(11)	Não				
morada	text	Não				
localidade	text	Não				
codigo_postal	varchar(8)	Não				
telefone	int(11)	Não				
email	text	Não				
website	text	Sim	NULL			
observacoes	text	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	empresas_ID	10	A	Não	
telefone	BTREE	Sim	Não	telefone	10	A	Não	
contribuinte	HASH	Sim	Não	contribuinte		A	Não	
				email		A	Não	
				website		A	Sim	
FK_empresas_noname_administrativo	BTREE	Não	Não	administrativo_ID	10	A	Não	

empresas_ramo

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
empresas_empresas_ID_ (Primária)	int(11)	Não		empresas -> empresas_ID		
ramo_ramo_ID_ (Primária)	int(11)	Não		ramo -> ramo_ID		

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	empresas_empresas_ID_	26	A	Não	
				ramo_ramo_ID_	26	A	Não	
FK_ramo_empresas_ramo_empresas_	BTREE	Não	Não	ramo_ramo_ID_	26	A	Não	

estabelecimentos

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
administrativo_ID	int(11)	Sim	NULL	administrativo -> utilizador_ID		
empresas_empresas_ID	int(11)	Não		empresas -> empresas_ID		
zona_zona_ID	int(11)	Não		zona -> zona_ID		
estabelecimentos_ID (Primária)	int(11)	Não				
nome	text	Não				
tipo_estabelecimento	text	Não				
morada	text	Não				
localidade	text	Não				
codigo_postal	varchar(8)	Não				
telefone	int(11)	Não				
email	text	Não				
foto	text	Sim	NULL			
horario	text	Sim	NULL			
data_fundacao	date	Não				
ja_aceitou_estagiarios	bit(1)	Sim	NULL			
observacoes	text	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	estabelecimentos_ID	20	A	Não	
telefone	HASH	Sim	Não	telefone		A	Não	
				email		A	Não	
FK_estabelecimentos_estabelecimentos_zona_zona	BTREE	Não	Não	zona_zona_ID	10	A	Não	
FK_estabelecimentos_noname_administrativo	BTREE	Não	Não	administrativo_ID	10	A	Sim	
FK_estabelecimentos_empresas	BTREE	Não	Não	empresas_empresas_ID	20	A	Não	

estabelecimentos_produtos

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
estabelecimentos_estabelecimentos_ID_ (Primária)	int(11)	Não		estabelecimentos -> estabelecimentos_ID		
produtos_produtos_ID_ (Primária)	int(11)	Não		produtos -> produtos_ID		

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	estabelecimentos_estabelecimentos_ID_	0	A	Não	
				produtos_produtos_ID_	0	A	Não	
FK_produtos_estabelecimentos_produtos_estabelecimentos_	BTREE	Não	Não	produtos_produtos_ID_	0	A	Não	

estabelecimentos_transportes

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
estabelecimentos_estabelecimentos_ID_ (Primária)	int(11)	Não		estabelecimentos -> estabelecimentos_ID		
transporte_transporte_ID_ (Primária)	int(11)	Não		transporte -> transporte_ID		

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	estabelecimentos_estabelecimentos_ID_	43	A	Não	
				transporte_transporte_ID_	43	A	Não	
FK_transporte_estabelecimentos_transportes_estabelecimentos_	BTREE	Não	Não	transporte_transporte_ID_	43	A	Não	

estagio

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
alunos_ID	int(11)	Não		alunos -> utilizador_ID		
formador_ID	int(11)	Não		formador -> utilizador_ID		
estabelecimentos_estabelecimentos_ID	int(11)	Não		estabelecimentos -> estabelecimentos_ID		
estagio_ID (Primária)	int(11)	Não				
data_inicio	datetime	Sim	NULL			
data_fim	datetime	Sim	NULL			
nota_dada_empresa	int(11)	Sim	NULL			
nota_dada_escola	int(11)	Sim	NULL			
nota_relatorio	int(11)	Sim	NULL			
nota_procura	int(11)	Sim	NULL			
nota_final	int(11)	Sim	NULL			
nota_dada_pelo_aluno	int(11)	Sim	NULL			
responsavel_id_responsavel	int(11)	Sim	NULL	responsavel -> id_responsavel		

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	estagio_ID	30	A	Não	
FK_estagio_noname_estabelecimentos	BTREE	Não	Não	estabelecimentos_estabelecimentos_ID	30	A	Não	
FK_estagio_responsavel_ID	BTREE	Não	Não	responsavel_id_responsavel	30	A	Sim	
FK_estagio_noname_alunos	BTREE	Não	Não	alunos_ID	30	A	Não	
FK_estagio_noname_formador	BTREE	Não	Não	formador_ID	15	A	Não	

formador

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
utilizador_ID (Primária)	int(11)	Não		utilizador -> utilizador_ID		
numero	int(11)	Não				
disciplina	text	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	utilizador_ID	7	A	Não	
numero	BTREE	Sim	Não	numero	7	A	Não	

produtos

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
produtos_ID (Primária)	int(11)	Não				
nome	text	Não				
marca	text	Não				

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
tipo_produto	text	Não				

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	produtos_ID	0	A	Não	

ramo

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
administrativo_ID	int(11)	Não		administrativo -> utilizador_ID		
ramo_ID (<i>Primária</i>)	int(11)	Não				
CAE	int(11)	Não				
descricao	text	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	ramo_ID	9	A	Não	
FK_ramo_noname_administrativo	BTREE	Não	Não	administrativo_ID	9	A	Não	

responsavel

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
id_responsavel (<i>Primária</i>)	int(11)	Não				
estabelecimentos_estabelecimentos_ID	int(11)	Não		estabelecimentos -> estabelecimentos_ID		
nome	text	Não				
título	text	Não				
cargo	text	Não				
telefone	int(11)	Não				
email	text	Não				
telemovel	int(11)	Não				
observacoes	text	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	id_responsavel	24	A	Não	
FK_responsavel_estabelecimentos	BTREE	Não	Não	estabelecimentos_estabelecimentos_ID	24	A	Não	

transporte

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
transporte_ID (<i>Primária</i>)	int(11)	Não				
meio_transporte	text	Não				
linha	text	Não				
observacoes	text	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	transporte_ID	17	A	Não	

turma

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
curso_curso_ID	int(11)	Não		curso -> curso_ID		
ano_letivo_ano_letivo_ID	int(11)	Não		ano_letivo -> ano_letivo_ID		
turma_ID (<i>Primária</i>)	int(11)	Não				
sigla	varchar(4)	Não				

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	turma_ID	18	A	Não	
FK_turma_noname_curso	BTREE	Não	Não	curso_curso_ID	6	A	Não	
FK_turma_noname_ano_letivo	BTREE	Não	Não	ano_letivo_ano_letivo_ID	4	A	Não	

utilizador

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
utilizador_ID <i>(Primária)</i>	int(11)	Não				
nome	text	Não				
login	text	Não				
password	text	Não				

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	utilizador_ID	361	A	Não	
login	HASH	Sim	Não	login		A	Não	

vagas

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
empresas_empresas_ID <i>(Primária)</i>	int(11)	Não		empresas -> empresas_ID		
ano_letivo_ano_letivo_ID <i>(Primária)</i>	int(11)	Não		ano_letivo -> ano_letivo_ID		
n_vagas	int(11)	Sim	NULL			
vagas_cheias	bit(1)	Sim	NULL			

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	empresas_empresas_ID	20	A	Não	
				ano_letivo_ano_letivo_ID	20	A	Não	
FK_vagas_noname_ano_letivo	BTREE	Não	Não	ano_letivo_ano_letivo_ID	4	A	Não	

zona

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
zona_ID <i>(Primária)</i>	int(11)	Não				
localidade	text	Não				
mapa	text	Sim	NULL			
designacao	text	Não				

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	zona_ID	5	A	Não	

zona_transportes

Coluna	Tipo	Nulo	Predefinido	Ligações para	Comentários	Tipo de mídia
zona_zona_ID_ <i>(Primária)</i>	int(11)	Não		zona -> zona_ID		
transporte_transporte_ID_ <i>(Primária)</i>	int(11)	Não		transporte -> transporte_ID		

Índices

Nome da chave	Tipo	Único	Pacote	Coluna	Quantidade	Agrupamento (Collation)	Nulo	Comentário
PRIMARY	BTREE	Sim	Não	zona_zona_ID_	17	A	Não	
				transporte_transporte_ID_	17	A	Não	
FK_transporte_zona_transportes_zona_	BTREE	Não	Não	transporte_transporte_ID_	17	A	Não	